

Day 34: Print Receipt Using f-string

Day 34 — Print Receipt Using f-string

1. Day Number

Day 34

2. Topic Name

f-string formatting

Today you will learn how to combine **text and variable values** neatly when printing output.

3. Connection

Yesterday, you calculated the **cart total and discount**.

The flow so far is:

```
Product
  ↓
Add to Cart
  ↓
Calculate Total
  ↓
Apply Discount
  ↓
Print Receipt ← Today
```

Today, instead of printing raw values separately, you will create a simple **shopping receipt**.

4. Important Topics

Today's main concepts are:

- f-strings
- variables
- output formatting

Example idea:

```
name = "Keyboard"

print(f"Product: {name}")
```

Output:

```
Product: Keyboard
```

The `f` before the string tells Python:

| Replace anything inside `{ }` with its value.

5. Foundational Notes

What is an f-string?

An **f-string** is a convenient way to put variables inside a string.

Without an f-string:

```
product = "Mouse"

print("Product:", product)
```

With an f-string:

```
print(f"Product: {product}")
```

Both can produce:

```
Product: Mouse
```

The f-string version becomes especially useful when there are several variables.

Basic syntax

```
f"some text {variable}"
```

For example:

```
age = 25

print(f"Age: {age}")
```

Output:

```
Age: 25
```

Multiple variables

You can use several variables in the same f-string.

```
item = "Book"
quantity = 2

print(f"Item: {item}, Quantity: {quantity}")
```

Output:

```
Item: Book, Quantity: 2
```

Expressions inside f-strings

Python can also evaluate simple expressions inside {}.

For example:

```
price = 100
quantity = 3

print(f"Total: {price * quantity}")
```

Output:

```
Total: 300
```

For today's problem, however, keep things simple by storing the total in a variable first.

Conceptually:

```
quantity × price
    ↓
    total
    ↓
print using f-string
```

Printing a receipt

A receipt is just several formatted lines.

Conceptually:

```
-----
      RECEIPT
-----
Product: Keyboard
Quantity: 2
Price: 500
Total: 1000
-----
```

Each line can be printed using an f-string when it contains a variable.

6. Easy Example

Suppose you have:

```
student = "Amit"
score = 85
```

You could display the information using:

```
print(f"Student: {student}")
```

```
print(f"Score: {score}")
```

Output:

```
Student: Amit  
Score: 85
```

The important idea is:

```
Variable  
  ↓  
{variable}  
  ↓  
f-string  
  ↓  
Formatted output
```

7. Problem Statement

Create a simple Python program that stores:

- product name
- quantity
- price
- total

Then print a simple receipt using **f-strings**.

For example, your receipt might conceptually look like:

```
----- RECEIPT -----  
Product: Keyboard  
Quantity: 2  
Price: 500  
Total: 1000  
-----
```

Your main goal today is **not complex calculation**.

Your main goal is learning how to display variables cleanly using f-strings.

8. Concepts Used

Variables

Store product information.

Conceptually:

```
product_name  
quantity
```

```
price
total
```

Arithmetic

The total normally represents:

```
total = quantity × price
```

Example:

```
2 × 500 = 1000
```

f-string

Used to combine labels and variable values.

Pattern:

```
f"Label: {variable}"
```

print()

Used to display each receipt line.

Example idea:

```
print heading
print product
print quantity
print price
print total
print closing line
```

9. Thought Process

Suppose the product is:

```
Product = Keyboard
Quantity = 2
Price = 500
```

First determine what information the receipt needs:

```
Product name
Quantity
Price
Total
```

Then think about the total:

2 × 500
↓
1000

Now store the information:

product_name → Keyboard
quantity → 2
price → 500
total → 1000

Finally, display those variables using f-strings:

Stored values
↓
f-strings
↓
Formatted receipt

So the overall thinking is:

Store product data
↓
Determine total
↓
Create receipt format
↓
Insert variables using { }
↓
Print receipt

10. Pseudocode

START

Create product name variable
Create quantity variable
Create price variable

Calculate or store total

Create a function for printing the receipt

Inside the function:

- print receipt heading
- print product name using f-string
- print quantity using f-string
- print price using f-string
- print total using f-string
- print receipt ending

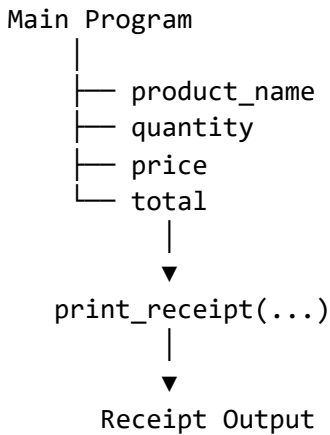
Call the function

END

11. Suggested Solving Approach — Functional Approach

Since you are practising the **functional approach**, separate the receipt-printing responsibility into a function.

Conceptually:



Think about a function like:

```
print_receipt(product_name, quantity, price, total)
```

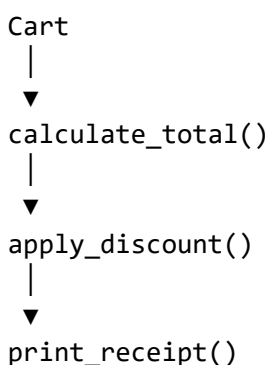
The function's responsibility should simply be:

Receive product information and display it as a receipt.

This is better than putting everything together because later your application might have functions such as:

```
calculate_total()
apply_discount()
print_receipt()
```

Eventually your shopping program could look conceptually like:



12. Easy Edge Cases

Edge Case 1 — Quantity is 0

Example:

```
Product: Keyboard  
Quantity: 0  
Price: 500  
Total: 0
```

Because:

```
0 x 500 = 0
```

Your program should still be able to print the receipt.

Edge Case 2 — Price is 0

Example:

```
Product: Free Sample  
Quantity: 2  
Price: 0  
Total: 0
```

Because:

```
2 x 0 = 0
```

Again, the receipt should still print normally.

13. Expected Input

For today's basic exercise, you do **not need** `input()` unless you want extra practice.

You can directly store values in variables.

Example values:

```
Product name: Keyboard  
Quantity: 2  
Price: 500  
Total: 1000
```

These values are then passed to your receipt-printing function.

14. Expected Output

For example:


```
-----  
      RECEIPT  
-----  
Product: Keyboard  
Quantity: 2  
Price: 500  
Total: 1000  
-----
```

Your exact decorative lines can be different.

The important requirement is that the values are printed using **f-strings**.

15. Hint Only

Start with four variables:

```
product_name  
quantity  
price  
total
```

Then create a function that accepts those four values.

Inside the function, remember the basic f-string pattern:

```
print(f"Label: {variable}")
```

Try to build each receipt line using this pattern.

For the total, think:

```
quantity × price
```

You now have everything needed to solve Day 34 without the full solution.